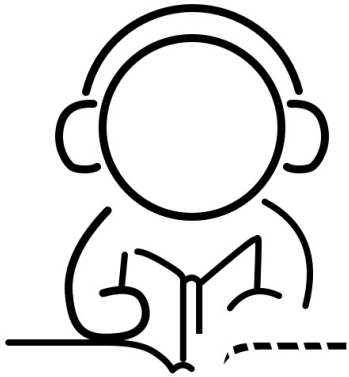
It's time for study

Daily Reminder



beautiful things happen
when you do the work to reprogram
that negative voice in your head

$a = 1$ → $t_0 = 1$, $a = t_0$

$a = b + c$ → $t_0 = b$, $t_1 = c$, $t_2 = t_0 + t_1$, $a = t_2$

$a = 2 + 3$ → $t_0 = 2$, $t_1 = 3$, $t_2 = t_0 + t_1$, $a = t_2$

arr[3] 3×4

(int) → arr

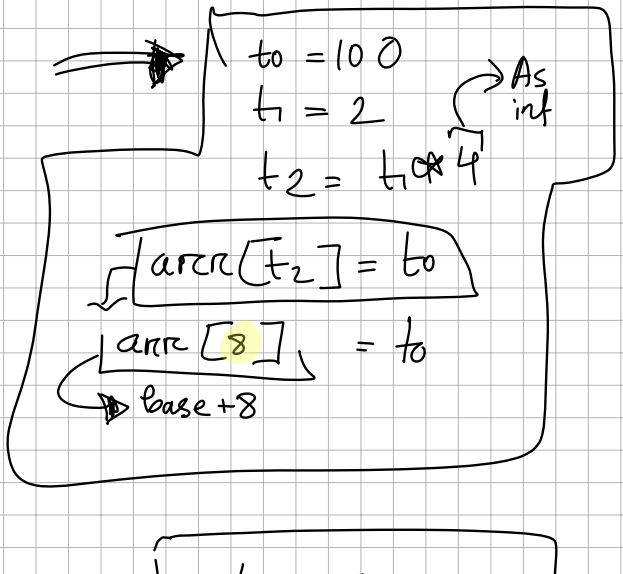
arr [2] = 8

arr[0]	[1]	[2]
100	103	108 — 111

100 + 2 × 4 = 108 (base, index, size of int)

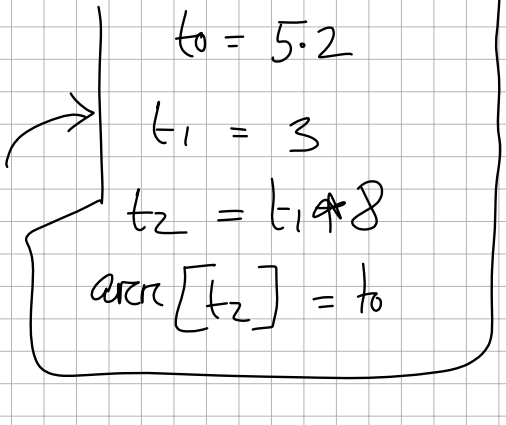
100	108
101	9
102	10
103	11

arr[2] = 100



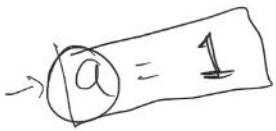
float arr[8];

arr[3] = 5.2;



void func(int a)

}



}

t0 = a

t1 = 1

t0 = t1

void func(int a)

}

int c;

c = ;

}

t0 = a

t1 = 2

c = t1

a = func(1, 2, 3);

t0 = 1

param t0

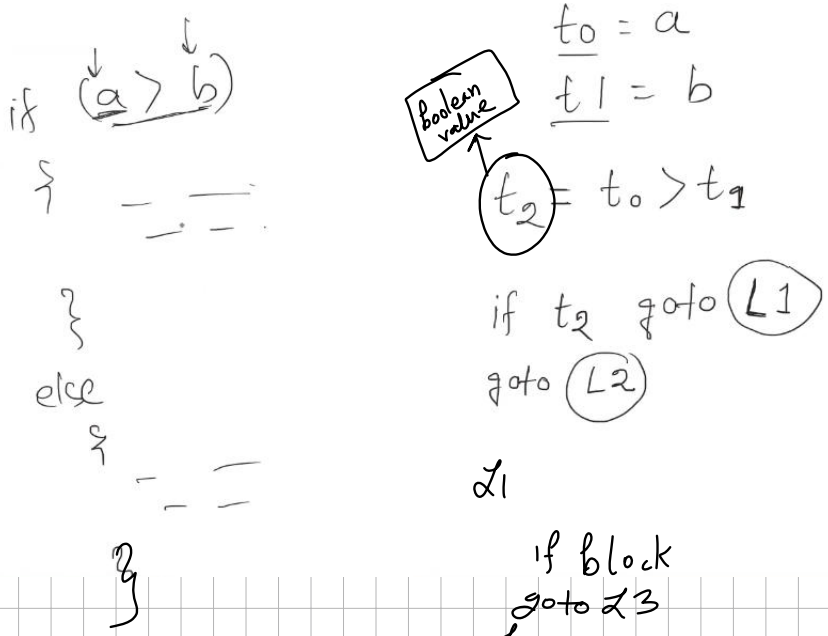
t1 = 2

param t1

param t2

t3 = call func, 3 (# of params)

a = t3


if (a > b)
{
}
else
{
}
t0 = a
t1 = b
boolean value
t2 = t0 > t1
if t2 goto L1
goto L2
L1
If block
goto L3
L2
else block
L3

```c
int func(int a, float b) {
    return a+b;
}
```

```
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: int func(int a, float b)
t0 = a
t1 = b
t2 = t0 + t1
return t2


//========== END OF CODE ==========
rakib@rakib-Lenovo-Legion-Y740-15IRHg:~/Downloads/La
tion-20260329T023720Z-3-001/Lab4-Three Address Code
```

Because `a` **and** `b` **are already function parameters.**

Look at the original C function:

```c
int func(int a, float b) {
    return a + b;
}
```

Here:

- `a` → parameter
- `b` → parameter

So when converting this function into **Three-Address Code (TAC)**, we don't need a separate `param` instruction just to declare them.

The TAC shown is:

```
t0 = a
t1 = b
t2 = t0 + t1
return t2
```

Then when do we use `param`?

`param` is normally used when **calling another function**, to pass arguments to that function.

For example:

```
x = func(5, 2.5);
```

TAC could be:

```
param 5
param 2.5
t0 = call func, 2
x = t0
```

```
1 void main()
2 {
3     int a;
4     int b;
5     a = 1;
6     b = 1;
7
8 }
```

```
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: void main()
// Declaration: int a
// Declaration: int b
t0 = 1
a = t0
t1 = 1
b = t1


//========== END OF CODE ==========
```

```
1 void main()
2 {
3    int a,b,c,d;
4    a = 1;
5    b = 2;
6    c = 3;
7    d = 4;
8    int sum;
9    sum = a + b + c + d;
10 }
```

```
// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: void main()
// Declaration: int a
// Declaration: int b
// Declaration: int c
// Declaration: int d
t0 = 1
a = t0
t1 = 2
b = t1
t2 = 3
c = t2
t3 = 4
d = t3
// Declaration: int sum
t4 = a
t5 = b
t6 = t4 + t5
t7 = c
t8 = t6 + t7
t9 = d
t10 = t8 + t9
sum = t10
```

```
1 int func(int a, int b)
2 {
3    int x;
4
5    x = 9;
6
7    return x;
8 }
9
10 void main()
11 {
12   func(1,2);
13 }
```

```
------------ Three Address Code ------------
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: int func(int a, int b)
// Declaration: int x
t0 = 9
x = t0
t1 = x
return t1

// Function: void main()
t2 = 1
param t2
t3 = 2
param t3
t4 = call func, 2


//========== END OF CODE ==========
```

Just Declaration of array but no use

```
1 void main()
2 {
3   int a;
4   int arr[10];
5 }
```

```
------------ Three Address Code ------------
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: void main()
// Declaration: int a
// Declaration: int arr[10]


//========== END OF CODE ==========
```

Actually using array

```
1 void main()
2 {
3   int a;
4   int arr[10];
5
6   arr[2] = 4;
7 }
```

```
------------ Three Address Code ------------
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: void main()
// Declaration: int a
// Declaration: int arr[10]
t0 = 4
t1 = 2
t2 = t1 * 4
arr[t2] = t0
```

```
1 void main()
2 {
3   int a;
4   a = 1;
5   int b;
6   b = 2;
7
8   int c;
9
10  if(a > b)
11  {
12      c = 3;
13  }
14  else
15  {
16      c = 5;
17  }
18 }
```

```
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

// Three Address Code

// Function: void main()
// Declaration: int a
t0 = 1
a = t0
// Declaration: int b
t1 = 2
b = t1
// Declaration: int c
t2 = a
t3 = b
t4 = t2 > t3
if t4 goto L0
goto L1
L0:
t5 = 3
c = t5
goto L2
L1:
t6 = 5
c = t6
L2:


//========== END OF CODE ==========
```

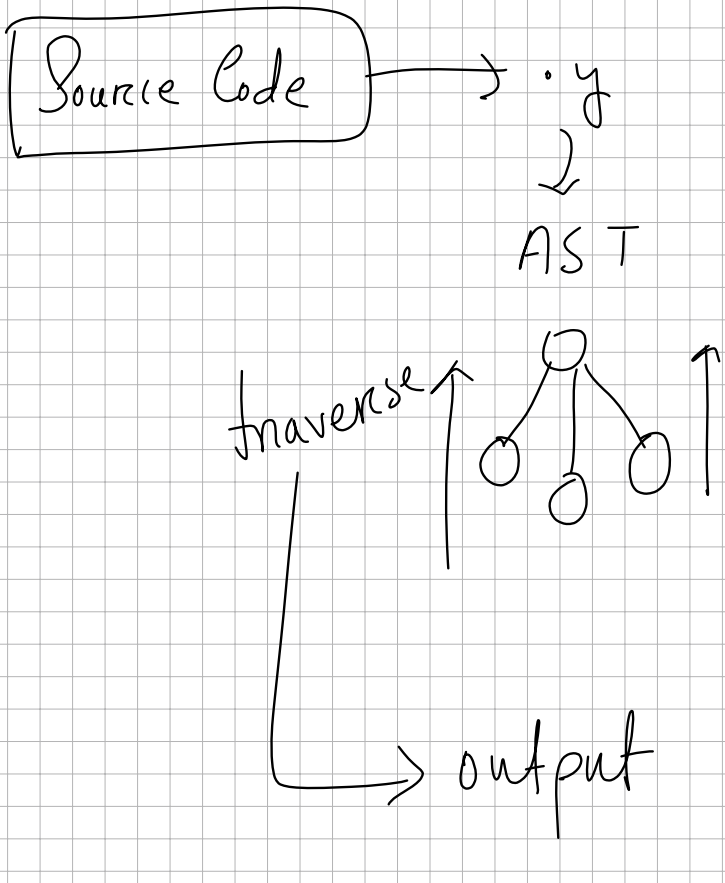
```
1 void main()
2 {
3
4   int i;
5   int a;
6
7   i = 1;
8
9   while(i < 100)
10  {
11      a = a + 1;
12  }
13 }
```

```
//========== THREE ADDRESS CODE ==========

// This code was generated by a two-pass compiler
// Format:
// - t0, t1, etc. are temporary variables
// - L0, L1, etc. are labels for jumps
// - Operations follow the three-address code format

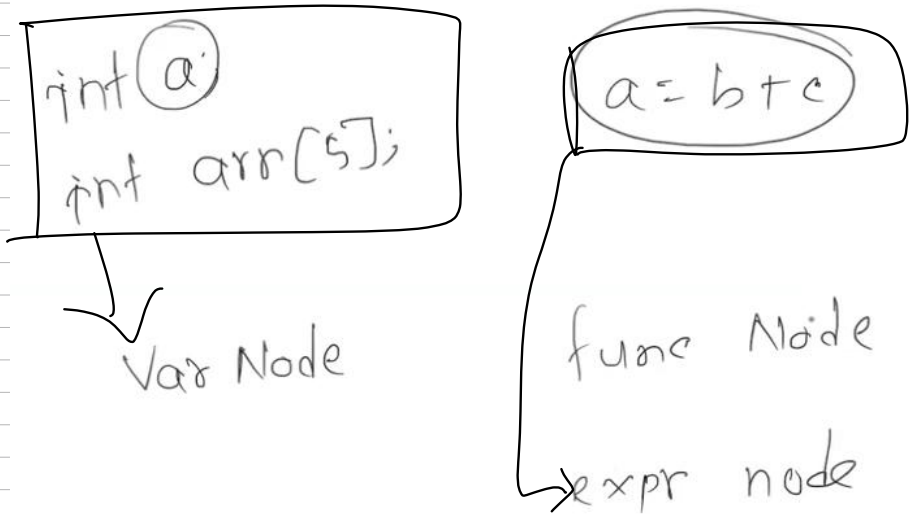
// Three Address Code

// Function: void main()
// Declaration: int i
// Declaration: int a
t0 = 1
i = t0
L0:
t1 = i
t2 = 100
t3 = t1 < t2
if t3 goto L1
goto L2
L1:
t4 = a
t5 = 1
t6 = t4 + t5
a = t6
goto L0
L2:
```


Source Code
.y
AST
traverse
output

Nodes in AST

written in specs



- **ConstNode** for constants
- **BinaryOpNode** for binary operations (+, -, *, /, etc.)
- **UnaryOpNode** for unary operations (-, !, etc.)
- **AssignNode** for assignment operations
- **FuncCallNode** for function calls

StmtNode classes for different statements:

- **ExprStmtNode** for expression statements
- **BlockNode** for compound statements (blocks)
- **IfNode** for if-else statements
- **WhileNode** for while loops
- **ForNode** for for loops
- **ReturnNode** for return statements
- **DeclNode** for variable declarations

ProgramNode as the root of the AST representing the entire progra